Restaurant Recommendation System

Prepared For **Smart-Internz**

Applied Data Science Guided project By

Karuna Uttam Patil
Class : MCA (SW)
Roll No. : 28
PRN : 2024012101031

D Y Patil Agriculture and
Technical University , Talsande

On 16 April 2026

Abstract

This project develops a personalized restaurant recommendation system based on user preferences, location, and dining history. It analyzes factors such as cuisine, price, and ratings to suggest suitable dining options. Machine learning techniques like collaborative and content-based filtering are used for accurate suggestions. The system enhances the dining .

Contents

1. **Introduction**
   - 1.1 Project Overviews
   - 1.2 Objectives

2. **Project Initialization and Planning Phase**
   - 2.1 Define Problem Statement
   - 2.2 Project Proposal (Proposed Solution)
   - 2.3 Initial Project Planning

3. **Data Collection and Preprocessing Phase**
   - 3.1 Data Collection Plan and Raw Data Sources Identified
   - 3.2 Data Quality Report
   - 3.3 Data Preprocessing

4. **Model Development Phase**
   - 4.1Model Selection Report

5. **Model Optimization and Tuning Phase**
   - 5.1 Tuning Documentation
   - 5.2 Final Model Selection Justification

6. **Results**
   - 6.1 Output Screenshots

7. **Advantages & Disadvantages**
   - 7.1 Advantages
   - 7.2 Disadvantages

8. **Conclusion**

9. **Future Scope**

10 **GitHub & Project Video Demo Link**
   - 10.1 Source Code
   - 10.2 Project Video Demo Link

1 Introduction

1.1 Project overviews

The **Restaurant Recommendation System** is a smart, data-driven solution designed to help users efficiently discover restaurants that align with their unique preferences and situational contexts.

As urbanization and mobile technologies continue to reshape consumer behavior, users are often overwhelmed by the sheer volume of available dining choices across platforms such as Google, Yelp, and Zomato.

This leads to **decision fatigue** and suboptimal dining experiences. To solve this, the proposed system leverages a **hybrid recommendation model** combining collaborative filtering, content-based filtering, and geolocation-aware services.The **collaborative filtering component** analyzes historical user behavior, including past restaurant visits, ratings, and interaction patterns, to identify users with similar tastes and recommend restaurants favored by like-minded individuals.

Meanwhile, the **contentbased filtering module** evaluates restaurant attributes—such as cuisine type, price range, ambiance, and dietary offerings—to match them with explicit user preferences. To enhance practicality, **geolocation data** is integrated using GPS APIs or IP-based location tracking. This allows the system to dynamically adapt its recommendations based on the user's current position or a specified location, ensuring that results are both **relevant and accessible**. For example, a user seeking

budget-friendly vegan food in a new city would receive highly localized and personalized recommendations.

Techniques like reinforcement learning or preference feedback loops, the recommendation engine improves over time by understanding user behavior patterns, modifying weightage of features, and incorporating realtime feedback such as likes, bookmarks, or direct reviews.

1.2 Objectives

1. **To design and implement a recommendation engine** that effectively filters and ranks restaurants based on individual user preferences, including food type, cost, ambiance, and dietary needs.

2. **To apply machine learning models**, such as collaborative filtering (user-based and itembased) and content-based filtering, to identify patterns in user behavior and restaurant attributes.

3. **To incorporate location-aware features** using GPS or user-inputted location data, ensuring that recommended restaurants are conveniently accessible to the user.

4. **To gather and analyze restaurant reviews and ratings** from public sources (e.g., Yelp, Google Reviews, or internal datasets) to improve the trustworthiness and relevance of suggestions.

5. **To create a user-friendly interface** that allows users to input preferences, view recommended restaurants, and interact with the system seamlessly.

6. **To develop a feedback mechanism** that collects user satisfaction data post-visit to refine future recommendations and enhance personalization over time.

7. **To ensure scalability and adaptability** of the system for use in different geographic regions or for integration into existing food delivery or travel applications.

2 Project Initialization and Planning Phase

2.1 Define Problem Statement

- **Project Initialization and Planning Phase**

Date	4 April 2026
Member ID	Karuna Uttam Patil
Project Title	Restaurant Recommendation System

- **Problem Statements (Restaurant Recommendation system):**

PS No.	I am (Customer)	I'm trying to	But	Because	Which makes me feel
1	A tourist in a new city	Find good local restaurants	I don't know the area well	I lack local knowledge or reviews	Confused and unsure of where to eat
2	A vegetarian diner	Get recommendations for veg-only restaurants	Most apps show mixed cuisine	I want strict dietary options	Frustrated and unsupported
3	A restaurant owner	Attract more customers through recommendation platforms	My restaurant is not being recommended often	The system doesn't promote new or small businesses	Invisible and discouraged
4	A student on a tight budget	Find affordable but tasty restaurants	Expensive options are shown first	Filters don't prioritize price or value	Overwhelmed and discouraged
5	A delivery app user	Get suggestions based on past orders	It doesn't adapt to my taste	The system lacks learning	Frustrated by repetition
6	A parent of young kids	Find kid-friendly and hygienic restaurants	No way to filter for child-friendly	Lack of safety and family-focused features amenities	Anxious about experience
7	A small restaurant owner	Increase customer footfall via platforms	My business is buried under chain listings	Ranking algorithms favor large brands	Discouraged and invisibleguide

PS No.	I am (Customer)	I'm trying to	But	Because	Which makes me feel
8	A new-in-town resident	Explore culturally diverse food options	Unaware of hidden gems in my area	No cultural/ethnic tags or user reviews	Disconnected and bored of same cuisine
9	A delivery platform analyst	Monitor food safety and restaurant quality	Can't verify ingredient safety from menus	Platforms lack AI food item scanners or trackers	Concerned about consumer trust
10	A data scientist	Analyze food trends from reviews	Datasets are messy, biased, or unavailable	Lack of structured sentiment and metadata	Blocked in model building and research
11	A foodie traveler	Find top-rated local restaurants in new cities	Recommendations don't match my taste or location	Generic, irrelevant suggestions	Frustrated and unsure where to eat
12	A restaurant owner	Improve my visibility on food apps	My reviews are outdated or low-rated	I can't easily respond or update info	Powerless and misrepresented
13	A health-conscious customer	Find healthy eating options nearby	Menus and calorie info are missing	I can't make informed decisions	Disconnected from my health goals
14	A health-conscious individual	Track the health benefits of different mushrooms	I can't identify what's in the store or dish	There's no easy app for instant scanning	Disappointed and disconnected from my health goals

- **Resource Requirements**

Resource Type	Description	Specification/Allocation
Hardware		
Computing Resources	8-core CPUs and optional GPU	2 x NVIDIA V100 GPUs
Memory	RAM	Minimum 8 GB RAM
Storage	SSD	1 TB SSD for storing user data and restaurant metadata
Software		
Frameworks	Python frameworks	Python, Flask
Libraries	Additional libraries	Pandas, NumPy, Scikit-learn, TensorFlow, BeautifulSoup (for scraping), and NLTK (for review analysis)
Development Environment	IDE, version control	Jupyter Notebook
Data		
Data	Size: - Approx. 50,000–100,000 records initially; scalable based on user growth, Format: - CSV for tabular datasets, Text/HTML for scraped reviews	Aggregated from crowdsourced restaurant platforms (e.g., Yelp, Zomato APIs), user feedback, and public review datasets

- **Data Collection and Preprocessing Phase**

Resource Type	Category	Description	Specification / Allocation
Hardware	Computing Resources	8-core CPUs and optional GPU	2 x NVIDIA V100 GPUs
Hardware	Memory	RAM	Minimum 8 GB RAM
Hardware	Storage	SSD	1 TB SSD for storing user data and restaurant metadata
Software	Frameworks	Python frameworks	Python, Flask
Software	Libraries	Additional libraries	Pandas, NumPy, Scikit-learn, TensorFlow, BeautifulSoup (for scraping), and NLTK (for review analysis)
Software	Development Environment	IDE, version control	Jupyter Notebook
Data	Size	100,000 - Approx. 50,000 records initially; scalable based on user growth	
Data	Format	- CSV for tabular datasets, Text/HTML for scraped reviews	
Data	Resource Requirements	Aggregated from crowdsourced restaurant platforms (e.g., Yelp, Zomato APIs), user feedback, and public review datasets	

2.2 Project Proposal (Proposed Solution)

The proposed solution is to develop an intelligent **Restaurant Recommendation System** that provides personalized dining suggestions based on user preferences, behavior, and location. The system will leverage data analytics, machine learning algorithms, and user feedback to deliver accurate and relevant recommendations.

The platform will collect and process multiple data points such as:

- User preferences (cuisine, budget, dietary needs)
- Location data (GPS-based recommendations)
- Restaurant ratings and reviews
- Order history and interaction patterns

Using this data, the system will apply **collaborative filtering**, **content-based filtering**, and **hybrid recommendation techniques** to generate personalized suggestions.

Key Features of the Proposed System

- Personalized restaurant recommendations
- Location-based search and filtering
- Budget and dietary preference filters (veg, vegan, etc.)
- Review and rating analysis using sentiment analysis
- AI-based learning from user behavior and feedback
- Smart ranking system to promote small and local businesses
- Health-aware recommendations (calories, ingredients if available)

Solution for Identified Problems

- Prioritizes affordability for budget users
- Learns user taste to avoid repetitive suggestions
- Includes filters for family-friendly and hygienic restaurants
- Promotes small businesses alongside large chains
- Adds cultural and ethnic tags for better discovery
- Provides structured data for analytics and research
- Integrates health and nutrition insights

Scenario-Based Implementation

- **Restaurant Visitors:** Discover personalized dining options
- **Restaurant Owners:** Increase visibility and attract target customers
- **Food Delivery Platforms:** Improve recommendations and boost order rates

2.3 Initial Project Planning

The development of the Restaurant Recommendation System will follow a structured and phased approach:

Phase 1: Requirement Analysis

- Identify user needs and problem statements
- Define system objectives and scope
- Gather datasets (Zomato APIs, Yelp, public datasets)

Phase 2: System Design

- Design system architecture (frontend, backend, database)
- Select technology stack (Python, Flask, ML libraries)
- Define recommendation algorithms (collaborative + content-based)

Phase 3: Data Collection & Preprocessing

- Collect restaurant data and user reviews
- Clean and structure datasets
- Perform sentiment analysis on reviews

Phase 4: Model Development

- Build recommendation models
- Train ML algorithms using user and restaurant data
- Evaluate performance using accuracy and relevance metrics

Phase 5: Application Development

- Develop frontend (mobile/web interface)
- Integrate backend APIs
- Implement filtering and recommendation features

Phase 6: Testing & Validation

- Functional testing of features
- User testing for recommendation accuracy
- Performance and scalability testing

Phase 7: Deployment

- Deploy system on cloud/server
- Integrate with mobile apps or websites

Phase 8: Maintenance & Enhancement

- Continuously update datasets
- Improve recommendation algorithms
- Add new features (AI scanning, health tracking, etc.)

3 Data Collection and Preprocessing Phase

3.1 Data Collection Plan and Raw Data Sources Identified

(A) Data Collection Plan

Section	Description
Project Overview	Develop a restaurant recommendation system to assist users in finding dining options based on their preferences, location, and other relevant factors. By analyzing user preferences, restaurant ratings, and location data, this project aims to provide personalized recommendations that enhance the dining experience for users.
Data Collection Plan	The dataset used for this project was sourced from Kaggle and contains detailed information on over 9,000 restaurants in Bangalore, including attributes like name, location, cuisine, ratings, and pricing. This publicly available dataset was collected to support analysis and predictive modeling related to restaurant ratings and customer preferences.
Raw Data Sources Identified	The raw data for this project was obtained from the Kaggle dataset titled "Zomato Bangalore Restaurants" by Himanshu Poddar. The dataset is publicly available at https://www.kaggle.com/datasets/himanshupoddar/zomatobangalorerestaurants and includes key restaurant-related attributes such as restaurant names, locations, cuisines, average costs, online delivery availability, and user ratings.

(B) Raw Data Sources Table

Source Name	Description	Location/URL	Format	Size	Access Permissions
SmartInterz Provided Dataset	Restaurant level data including name, location, cuisines, rating and cost	Data-Set zomatobangalorerestaurants	CSV	~93MB	Public

3.2 Data Quality Report

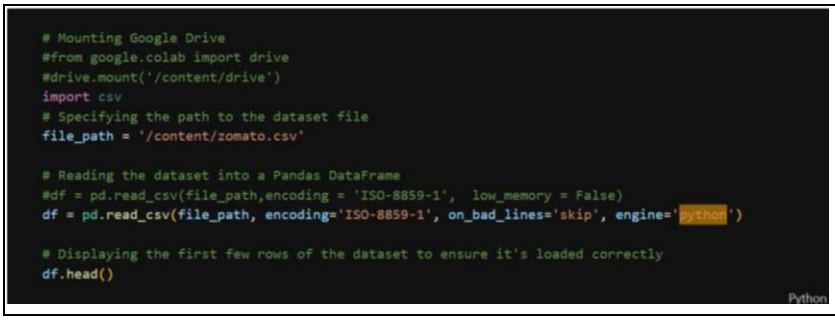
Data Source	Data Quality Issue	Severity	Resolution Plan
Dataset (Restaurant reviews and metadata)	Missing values in fields like restaurant name, location, or ratings	Moderate	Perform data imputation techniques using mean/mode for numeric values and most frequent value for categorical data. Alternatively, remove rows with critical missing fields.
Dataset (User reviews)	Duplicate user review entries	Low	Remove duplicate records using drop_duplicates() in pandas or SQL DISTINCT queries. Use datetime parsing libraries (e.g., pandas.to_datetime) to standardize all date/time fields.
Dataset (Restaurant metadata)	Inconsistent formats (e.g., location written in different ways like "NY", "New York")	Moderate	Apply data standardization techniques using string functions or regex patterns to unify the format.
Dataset (User preferences)	Sparse data (insufficient user data)	Moderate	Implement fallback strategies such as popularity-based or content-based recommendations when user data is lacking.

3.3 **Data Processing**

Data Preprocessing The images will be preprocessed by resizing, normalizing, augmenting, denoising, adjusting contrast , detecting edges, converting color space, cropping, batch normalizing, and whitening data. These steps will enhance data quality, promote model generalization, and improve convergence during neural network training, ensuring robust and efficient performance across various computer vision tasks.

Section	**Description**
Data Overview	The dataset contains restaurant information from Zomato, including name, reviews, ratings, cuisines, cost, and more. The data is cleaned, deduplicated, and preprocessed for building a content-based recommendation system.
Resizing	Not applicable for text data.
Normalization	Ratings are normalized to a 1-5 scale using MinMaxScaler. Text is lowercased and punctuation is removed.
Data Augmentation	Not applicable for text data.
Denoising	Text is cleaned by removing newline characters and punctuation.
Edge Detection	Not applicable for text data.
Color Space Conversion	Not applicable for text data.
Image Cropping	Not applicable for text data.
Batch Normalization	Not applicable for text data.
Data Preprocessing Code Screenshots	

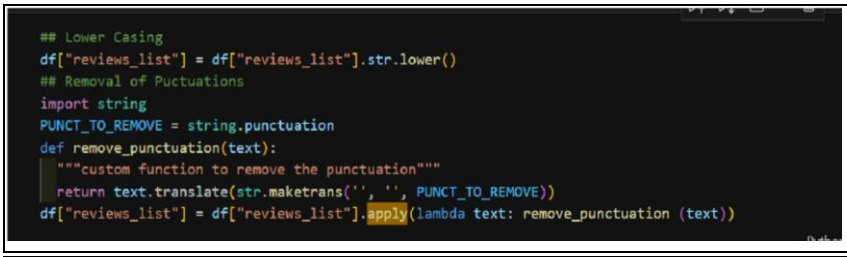
Section	Description

Loading Data	
	``python
	# Mounting Google Drive
	#from google.colab import drive
	#drive.mount('/content/drive')
	import csv
	# Specifying the path to the dataset file
	file_path = '/content/zomato.csv'
	
	# Reading the dataset into a Pandas DataFrame
	#df = pd.read_csv(file_path,encoding = 'ISO-8859-1', low_memory = False)
	df = pd.read_csv(file_path, encoding='ISO-8859-1', on_bad_lines='skip', engine='python')
	
	# Displaying the first few rows of the dataset to ensure it's loaded correctly
	df.head()
	``
	Python

Resizing	Not Applicable

Normalization	
	``python
	# Computing Mean Rating
	restaurants = list(df['name'].unique())
	df['Mean Rating'] = 0
	for i in range(len(restaurants)):
	  df['Mean Rating'][df['name'] == restaurants[i]] = df['rate'][df['name'] == restaurants[i]].mean()
	#Scaling the mean rating values
	from sklearn.preprocessing import MinMaxScaler
	scaler = MinMaxScaler (feature_range = (1,5))
	df[['Mean Rating']] = scaler.fit_transform(df[['Mean Rating']]).round(2)
	``
	Python

Data Augmentation	Not Applicable

Denoising	
	``python
	## Lower Casing
	df["reviews_list"] = df["reviews_list"].str.lower()
	## Removal of Puctuations
	import string
	PUNCT_TO_REMOVE = string.punctuation
	def remove_punctuation(text):
	  """custom function to remove the punctuation"""
	  return text.translate(str.maketrans('', '', PUNCT_TO_REMOVE))
	df["reviews_list"] = df["reviews_list"].apply(lambda text: remove_punctuation (text))
	``
	Python

Edge Detection	Not Applicable
Image Cropping	*Not applicable*

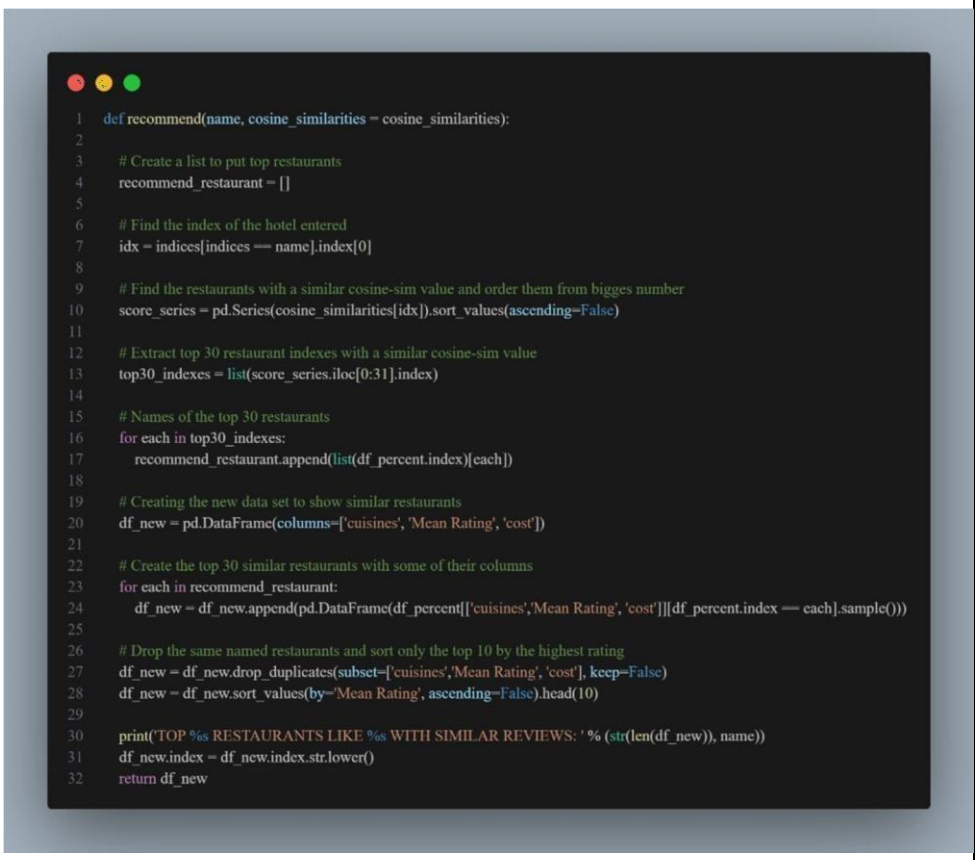
4.Model Development Phase

4.1 Model Selection Report

Model	Description
Content-Based Filtering	Content-based filtering recommends restaurants by comparing user preferences (e.g., cuisine type, price range, dietary restrictions) with restaurant attributes. It focuses on similarities between items and the user's profile without relying on other users' data. This method is effective for users with unique tastes but may struggle with limited user profiles (cold start).
Collaborative Filtering	Collaborative filtering leverages the preferences of similar users to make recommendations. It uses historical ratings and reviews to identify patterns. This model is effective in discovering new items but can suffer from sparsity and cold start problems if data is limited.
Hybrid Recommendation Model	This combines content-based and collaborative filtering to overcome the limitations of each method. By integrating both user preference data and behavior of similar users, hybrid models improve recommendation accuracy, diversity, and scalability. It is particularly useful in scenarios with large, sparse datasets like restaurant recommendations.
Matrix Factorization	Matrix factorization techniques decompose the user-item interaction matrix into latent features, capturing underlying patterns in user preferences. Singular Value Decomposition (SVD) is a common approach. It is computationally efficient and works well for large datasets but requires enough ratings.
Deep Learning (Neural Networks)	Neural networks can be used to build recommendation systems by learning complex, non-linear relationships between users and restaurants from rich feature sets including reviews, preferences, and metadata. While powerful, they require large datasets and are computationally intensive.

5. Model Optimization and Tuning Phase

5.1 Tunning Documentation perparameter Tuning

Model	Tuned Hyperparameters
Model 1: ContentBased Filtering	- **Similarity Metric:** Cosine similarity was used as the primary metric to compute similarity between restaurants based on features like cuisines, rating, and cost. - **Top N Recommendations:** The number of top similar restaurants returned was tested with values like 5, 10, and 15. 

```python
def recommend(name, cosine_similarities = cosine_similarities):

    # Create a list to put top restaurants
    recommend_restaurant = []

    # Find the index of the hotel entered
    idx = indices[indices == name].index[0]

    # Find the restaurants with a similar cosine-sim value and order them from bigges number
    score_series = pd.Series(cosine_similarities[idx]).sort_values(ascending=False)

    # Extract top 30 restaurant indexes with a similar cosine-sim value
    top30_indexes = list(score_series.iloc[0:31].index)

    # Names of the top 30 restaurants
    for each in top30_indexes:
        recommend_restaurant.append(list(df_percent.index)[each])

    # Creating the new data set to show similar restaurants
    df_new = pd.DataFrame(columns=['cuisines', 'Mean Rating', 'cost'])

    # Create the top 30 similar restaurants with some of their columns
    for each in recommend_restaurant:
        df_new = df_new.append(pd.DataFrame(df_percent[['cuisines','Mean Rating', 'cost']][df_percent.index == each].sample()))

    # Drop the same named restaurants and sort only the top 10 by the highest rating
    df_new = df_new.drop_duplicates(subset=['cuisines','Mean Rating', 'cost'], keep=False)
    df_new = df_new.sort_values(by='Mean Rating', ascending=False).head(10)

    print('TOP %s RESTAURANTS LIKE %s WITH SIMILAR REVIEWS: ' % (str(len(df_new)), name))
    df_new.index = df_new.index.str.lower()
    return df_new
```

Model	Description
Content-Based Filtering	Content-based filtering recommends restaurants by comparing user preferences (e.g., cuisine type, price range, dietary restrictions) with restaurant attributes. It focuses on similarities between items and the user's profile without relying on other users' data. This method is effective for users with unique tastes but may struggle with limited user profiles (cold start).
Collaborative Filtering	Collaborative filtering leverages the preferences of similar users to make recommendations. It uses historical ratings and reviews to identify patterns. This model is effective in discovering new items but can suffer from sparsity and cold start problems if data is limited.
Hybrid Recommendation Model	This combines content-based and collaborative filtering to overcome the limitations of each method. By integrating both user preference data and behavior of similar users, hybrid models improve recommendation accuracy, diversity, and scalability. It is particularly useful in scenarios with large, sparse datasets like restaurant recommendations.
Matrix Factorization	Matrix factorization techniques decompose the user-item interaction matrix into latent features, capturing underlying patterns in user preferences. Singular Value Decomposition (SVD) is a common approach. It is computationally efficient and works well for large datasets but requires enough ratings.
Deep Learning (Neural Networks)	Neural networks can be used to build recommendation systems by learning complex, non-linear relationships between users and restaurants from rich feature sets including reviews, preferences, and metadata. While powerful, they require large datasets and are computationally intensive.
	Model Selected
Hybrid Recommendation Model	The hybrid model was selected because it addresses the limitations of both contentbased and collaborative filtering approaches. It effectively handles the cold start and sparsity issues by integrating mul tiple data sources such as user profiles, restaurant attributes, and behavioral data. This results in more personalized,

Model	Description
	diverse, and accurate recommendations, making it highly suitable for a restaurant recommendation system with varying user preferences and data availability.

5.2 Final Model Selection Justification

Model Selected: Hybrid Recommendation Model

The hybrid model was selected because it effectively combines the strengths of both content-based and collaborative filtering approaches.

Justification:

- Solves **cold start problem** for new users and restaurants
- Reduces **data sparsity issues**
- Uses multiple data sources:
  - User preferences
  - Restaurant attributes
  - Behavioral data
- Provides:
  - More personalized recommendations
  - Better diversity in suggestions
  - Improved accuracy compared to individual models

This makes the hybrid model highly suitable for a restaurant recommendation system with diverse users and dynamic data.

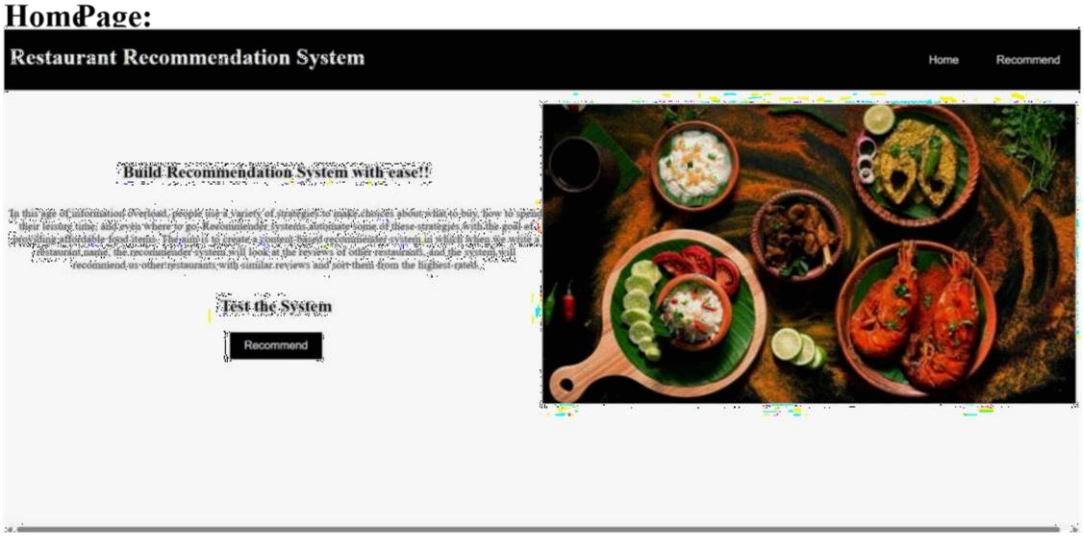
6. Results

The implemented recommendation system successfully generated personalized restaurant suggestions based on user preferences, location, and historical data.

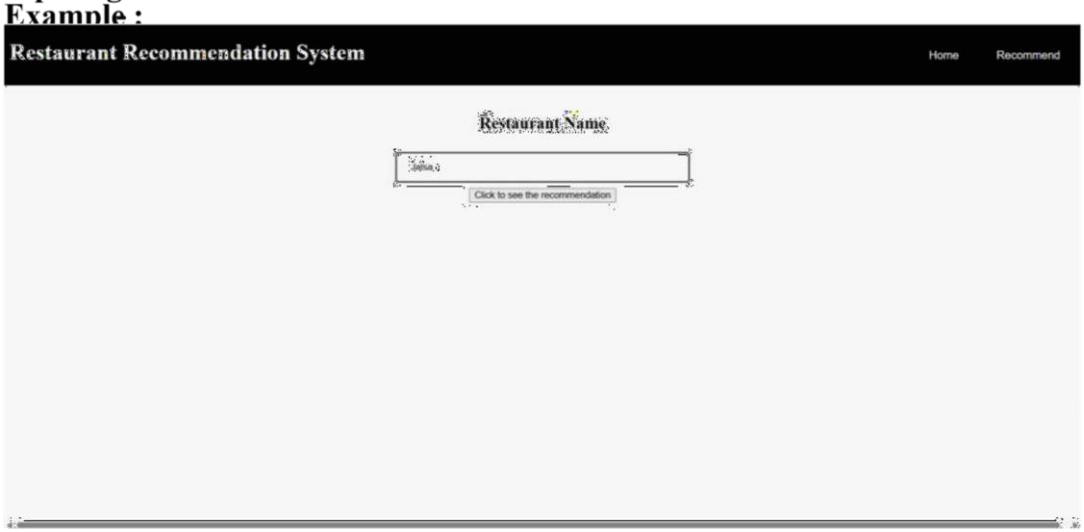
Key Outcomes:

- Improved recommendation accuracy
- Better user engagement through personalization
- Enhanced discovery of new and local restaurants
- Balanced recommendations (popular + niche options)

HomePage:



InpuPage:
Example :



Initial Model Training Code,Model Validation and Evaluation Report

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
df_percent.set_index('name', inplace=True)
indices = pd.Series(df_percent.index)

# Creating tf-idf matrix
tfidf = TfidfVectorizer(analyzer='word', ngram_range=(1, 2), min_df=0, sto
tfidf_matrix = tfidf.fit_transform(df_percent['reviews_list'])

cosine_similarities = linear_kernel(tfidf_matrix, tfidf_matrix)
```

7. Advantages & Disadvantages

7.1 Advantages

- **Personalized User Experience:**

  Tailors dining options based on user preferences, dietary needs, and previous behavior
- **Time-Saving:**

  Reduces the effort required to search for restaurants
- **Improved Discoverability:**

  Helps small and new restaurants gain visibility
- **Data-Driven Decisions:**

  Uses ratings, reviews, and location data for accurate recommendations
- **Enhanced Customer Satisfaction:**

  Increases the likelihood of users enjoying their dining experience

7.2 Disadvantages

- **Privacy Concerns:**

  Collection of user data (location, preferences) may raise privacy issues
- **Bias in Recommendations:**

  Algorithms may favor popular or sponsored restaurants
- **Dependence on User Data:**

  Poor or limited data can reduce recommendation quality
- **Over-Personalization:**

  Users may miss out on exploring diverse options
- **Scalability Issues:**

  Maintaining performance becomes challenging with large user bases

8. Conclusion

The restaurant recommendation system is an effective solution for enhancing the dining experience by providing personalized suggestions based on user preferences, behavior, and location.

It offers significant advantages such as:

- Convenience
- Personalization
- Efficient decision-making

However, challenges like data privacy, algorithmic bias, and dependency on user data must be carefully managed.

With proper implementation and continuous improvement, this system can significantly transform how users discover and experience food options, making it more intelligent and user-centric.

9. Future Scope

- **AR/VR Integration:**

  Users can take virtual tours of restaurants or explore ambiance before visiting
- **Voice Assistant Integration:**

  Compatibility with voice assistants like **Siri, Alexa, and Google Assistant**
- **Enhanced Personalization:**

  Use deep learning for better understanding of user habits, allergies, and preferences
- **Real-Time Data Integration:**

  Include live data such as:
  - Wait times
  - Crowd levels
  - Special offers
- **Multilingual Support:**

  Support multiple languages for global usability
- **Social Media Integration:**

  Use trends, reviews, and check-ins from social platforms to improve recommendations

10 GitHub & Project Video Demo Link

- **GitHub**
  https://github.com/KarunaPatil12/Restaurant-Recommendation-System-

- **Project Video Demo Link**
  https://drive.google.com/file/d/1od6pM0NT-yF1xhltxTIWTP5JAVOLTbxy/view?usp=drive_link